

Day 02: React Frontend Development (Static UI & Navigation Layer)

By the end of this session, learners will be able to:

- Design a modular frontend architecture using React.js
- Implement component-based UI design
- Establish routing and navigation flow
- Integrate API-ready frontend using Axios (conceptually)
- Simulate authentication flow using JWT (UI-level)
- Manage UI state effectively
- Build a fully navigable frontend using static data

Frontend Architecture Overview

Frontend architecture in modern systems is designed to ensure:

- Separation of Concerns (SoC)
- Reusability of components
- Scalability of UI modules

Unlike simple UI development, this phase focuses on **system-driven UI engineering**, where:

- UI is mapped to business roles
- Components are designed for reuse across modules
- Navigation is aligned with user journeys

Layered Frontend Design:

- UI Layer (Components)
- Routing Layer
- State Layer
- Service Layer (API abstraction)

Academic Insight

A well-structured frontend mimics backend architecture by enforcing **modularity and maintainability**.

React Project Architecture

A scalable React project follows a **feature-based or layered structure**.

Recommended Structure

```
src/  
├── components/  
├── pages/  
├── routes/  
├── services/  
├── context/  
├── utils/  
└── App.jsx
```

Conceptual Justification:

- components/ → reusable UI elements
- pages/ → screen-level components
- services/ → API abstraction layer
- context/ → state sharing

Academic Insight:

This structure aligns with **modular design principles** and improves **code maintainability index**.

React Components Design

A component is an independent, reusable UI unit.

Types of Components

- Functional Components (preferred)
- Presentational vs Container components

Core Application Components

Module	Purpose
Auth Component	Login/Signup UI
Dashboard	Entry point after login
Movie Catalog	Display movie list
Booking component	Handle selection flow

Example:

```
function MovieCard({ title, rating }) {
  return (
    <div>
      <h3>{title}</h3>
      <p>{rating}</p>
    </div>
  );
}
```

Component Classification

Type	Purpose
Atomic Components	Buttons, Input, Cards, etc.
Composite Components	Movie Card, Seat Layout, etc.
Page Components	Dashboard, Booking Page, etc.
Layout Component	Navbar, Sidebar, etc.

RBAC (Role Based Access Control) – UI Perspective

Defined Roles – User, Theatre Owner, Admin

Permission Matrix

Feature	User	Theatre Owner	Admin
View Movies	Yes	Yes	Yes
Book Tickets	Yes	No	No
Manage Shows	No	Yes	Yes
Add Movies	No	Yes	Yes
Manage Users	No	No	Yes
View Reports	No	Yes	Yes

Conceptual Implementation

- Role stored (static for now)
- UI rendered conditionally

Example

```
if (role === "admin") {
  return <AdminDashboard />;
}
```

Academic Insight: RBAC improves security, usability, and system clarity

React Routing & Navigation

Routing enables **Single Page Application (SPA)** behavior.

Library Used: react-router

Concept: URL ↔ Component mapping

Example:

```
<Route path="/dashboard" element={<Dashboard />} />
```

Navigation Flow Design

Defined Flow: Login → Dashboard → Movie Catalog → Booking

Design Consideration

- Logical user journey
- Minimal navigation friction

Insight: Navigation design impacts user experience (UX) and system usability metrics.

Protected Routes (Conceptual Implementation)

Protected routes restrict access based on authentication state.

Concept: If user is not authenticated → redirect to login

Example

```
function ProtectedRoute({ children }) {  
  const isAuthenticated = true; // static for now  
  return isAuthenticated ? children : <Navigate to="/login" />;  
}
```

Axios Integration (API Layer Preparation)

Axios is used for API communication.

Purpose

- Abstract API calls
- Prepare frontend for backend integration

Example

```
axios.get("/api/movies").then(res => console.log(res.data));
```

Design Insight

Even when using static data, defining API structure ensures:

- Smooth backend integration later
- Separation between UI and data logic

JWT Authentication Integration (Frontend Perspective)

JWT is handled at UI level by:

- Storing token (localStorage/sessionStorage)
- Sending token in requests (conceptually)

Flow (UI Simulation)

Login → Receive Token (mock) → Store → Use for navigation control

Security Note (Academic Context):

Frontend **should never fully trust client-side authentication**, validation must always occur at backend.

State Management Basics

State refers to dynamic data controlling UI behavior.

Types of State

- Local State (component-level)
- Global State (shared across components)

Example

```
const [movies, setMovies] = useState([]);
```

Conceptual Importance

State management ensures:

- Reactive UI updates
- Data consistency

Handling UI State

Common UI States

- Forms (input handling)
- Lists (rendering collections)
- Selection (user interaction)

Example (Form Handling)

```
const [email, setEmail] = useState("");
```

Design Principle

UI state must be:

- Predictable
- Controlled
- Isolated

Component Reusability & Clean Architecture

Reusability Principle: Build once, use multiple times

Example

- Button component used across all screens
- Card component reused for movies

Clean Architecture in Frontend

- Separate UI from logic
- Avoid tightly coupled components
- Follow DRY (Don't Repeat Yourself)

Static Data-Driven Development

At this stage:

- No backend integration
- Data is hardcoded or mocked

Why Static First?

- Focus on UI correctness
- Reduce dependency on backend readiness
- Improve parallel development

Industry Practice Insight:

Frontend teams often build UI using mock data before API readiness, ensuring faster development cycles.

Learning Outcomes

Students will be able to:

- Design modular React applications
- Implement routing and navigation logic
- Simulate authentication flow
- Manage UI state effectively
- Develop reusable components

Deliverables of Day 02

- All UI screens implemented using static data
- Navigation fully functional
- Protected routes conceptually implemented
- API-ready structure using Axios
- Clean and modular component structure

Viva Questions

1. What is component-based architecture in React?
2. Differentiate between functional and class components.
3. What is the role of React Router?
4. Explain the concept of SPA (Single Page Application).
5. What are protected routes and why are they needed?

6. What is Axios and how is it used?
7. How does frontend handle JWT tokens?
8. What is state in React?
9. Differentiate between local and global state.
10. What are controlled components in React?
11. Explain component reusability with examples.
12. What is the significance of project structure in React?
13. Why is static data used before API integration?
14. What is the difference between props and state?
15. How does navigation flow affect user experience?
16. What are best practices for frontend architecture?
17. Explain DRY principle in frontend development.
18. What are the limitations of client-side authentication?